



## Sistemas Digitais Avançados

Prof. Imbiriba

### **Mux\_Demux de 4 canais de 8 bits usando comandos concorrentes e sequenciais em VHDL**

#### **INTRODUÇÃO**

Os circuitos digitais multiplex e demultiplex são amplamente utilizados em sistemas de comunicação, incluindo transmissão de dados e voz, sistemas de televisão a cabo e satélite, sistemas de radar, entre outros. Um multiplexador é usado para combinar vários sinais de entrada em um único canal de transmissão, enquanto um demultiplexador é usado para separar o sinal combinado em vários sinais de saída individuais.

Por exemplo, em um sistema de transmissão de televisão a cabo, vários canais de televisão são combinados em um único cabo de transmissão usando um multiplexador. Quando o sinal chega à casa do assinante, um demultiplexador é usado para separar os diferentes canais de televisão para que possam ser exibidos em televisores separados.

Outra aplicação comum de multiplexadores e demultiplexadores é em sistemas de comunicação de dados, como redes de computadores. Um multiplexador pode ser usado para combinar vários sinais de entrada de dados em um único canal de transmissão, enquanto um demultiplexador pode ser usado para separar os sinais individuais de dados na extremidade receptora.

Nesse experimento iremos realizar um experimento que ilustra a aplicação de um circuito multiplexado de 4 canais, onde cada canal possui 8 bits. O descrição VHDL do circuito apresenta vários modelos de arquitetura para cada um dos circuitos: multiplexador-MUX e demultiplexador-DEMUX. Os modelos de arquitetura os circuitos multiplex e demultiplex são apresentados com níveis de abstração diferente, como por exemplo: modelo RTL (a nível de transferência entre registradores) e comportamental (utilizando PROCESS). Para a definição do modelo de arquitetura para entidade de mais alta nível (*top level*) será utilizado um modelo estrutural, utilizando as entidades MUX e DEMUX descritas em VHDL. A Figura1 ilustra o esquemático do projeto mux\_demux de 4 canais.

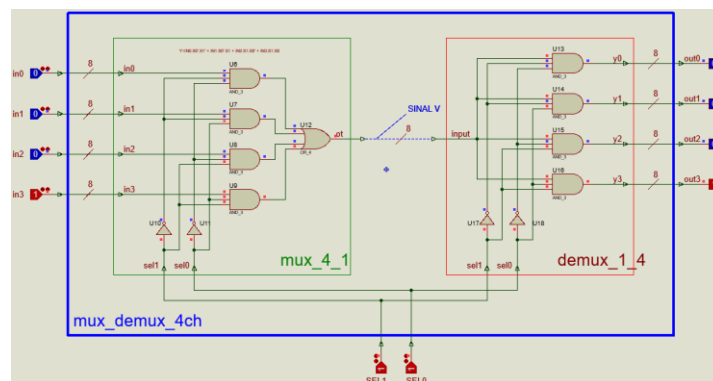


Figura1 – Esquemático da estrutura mux\_demux\_4ch de 8 bits



No experimento são introduzidos na prática alguns conceitos visto em aula, como o uso da biblioteca da IEEE [IEEE.std\_logic\_1164], a definição de uma constante genérica [generic], a definição de um sinal interno de uma entidade [signal], e o uso dos comandos concorrentes [when-else, with-select] e sequenciais [if-then-else, case-when] da linguagem VHDL.

## PARTE 1 - Descrição do MUX de 4 canais.

- 1- No quartus II vá em **File->New->VHDL** para criar um novo arquivo VHDL e edit o código da Figura 2. Salve o arquivo com o nome **mux\_4\_1.vhd**. Esse código VHDL implementa a descrição do circuito multiplexador de 4 canais com N bits.

```
1  -- std_logic ->
2  -- 'U': não inicializado. Este sinal ainda não foi definido.
3  -- 'X': desconhecido. Impossível determinar o valor / resultado.
4  -- '0': lógico 0
5  -- '1': lógica 1
6  -- 'Z': Alta Impedância
7  -- 'W': Sinal fraco, não é possível dizer se é 0 ou 1
8  -- 'L': Sinal fraco que provavelmente deve ser 0
9  -- 'H': Sinal fraco que provavelmente deve ser 1
10 -- '-': Não importa.
11
12 library IEEE;
13 use IEEE.std_logic_1164.all;
14
15 entity mux_4_1 is
16     generic (N : integer := 8); -- É um meio específico de passar informações entre entidades de projeto
17     -- Quando usando o modelo estrutural esse parâmetro pode ser passado por
18     -- entidades.
19
20     port (in0,in1,in2,in3 : in  STD_LOGIC_VECTOR (N-1 downto 0);
21           sel : in  STD_LOGIC_VECTOR (1 downto 0);
22           ot : out  STD_LOGIC_VECTOR (N-1 downto 0));
23 end mux_4_1;
24
25 -----usando modelo de fluxo de dados ou algoritmo com "WHEN ELSE" --> Atribuição
26 -----condicional de sinais
27 architecture rtl of mux_4_1 is
28     begin
29         ot <= in0 when sel = "00" else
30             in1 when sel = "01" else
31             in2 when sel = "10" else
32             in3;
33     end rtl;
34
35
36 ----- usando modelo de fluxo de dados ou algoritmo com "WITH SELECT" --> Atribuição
37 ----- de sinal com escolha
38 architecture rtl_with_select of mux_4_1 is
39     -- signal select_ch : bit_vector (1 downto 0);
40     begin
41         -- select_ch <= s1 & s0;
42         with sel select
43             ot <= in0 when "00",
44                 in1 when "01",
45                 in2 when "10",
46                 in3 when "11",
47                 null when others;
48     end rtl_with_select;
49
50
51 ----- usando if-then-else -----
52 architecture comportamental of mux_4_1 is
53     begin
54         mux: process (sel,in0,in1,in2,in3)
55             begin
56                 if sel = "00" then
57                     ot <= in0;
58                 elsif sel = "01" then
59                     ot <= in1;
60                 elsif sel = "10" then
61                     ot <= in2;
62                 elsif sel = "11" then
63                     ot <= in3;
64                 end if;
65             end process;
66         end comportamental;
67
68
69 ----- usando case-when -----
70 architecture comportamental2 of mux_4_1 is
71     begin
72         mux: process (sel,in0,in1,in2,in3)
73             begin
74                 case sel is
75                     when "00" => ot <= in0;
76                     when "01" => ot <= in1;
77                     when "10" => ot <= in2;
78                     when "11" => ot <= in3;
79                 end case;
80             end process;
81         end comportamental2;
```

Figura 2 – Código VHDL da descrição do MUX de 4 canais de N bits



- 2- Utilize as formas de onda de entrada da Figura 3 para simular o circuito implementado no código VHDL da Figura 2.

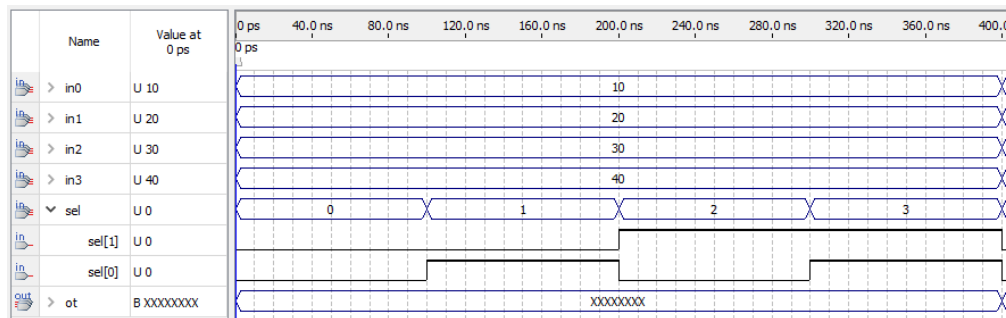


Figura 3 – Formas de onda de entrada VWF.

- 3- Realiza a simulação do circuito e veja as formas de onda de saída na ferramenta University Program VWF. Capture a tela de saída do resultado da simulação. A captura deve ser similar ao apresentado na Figura 4.

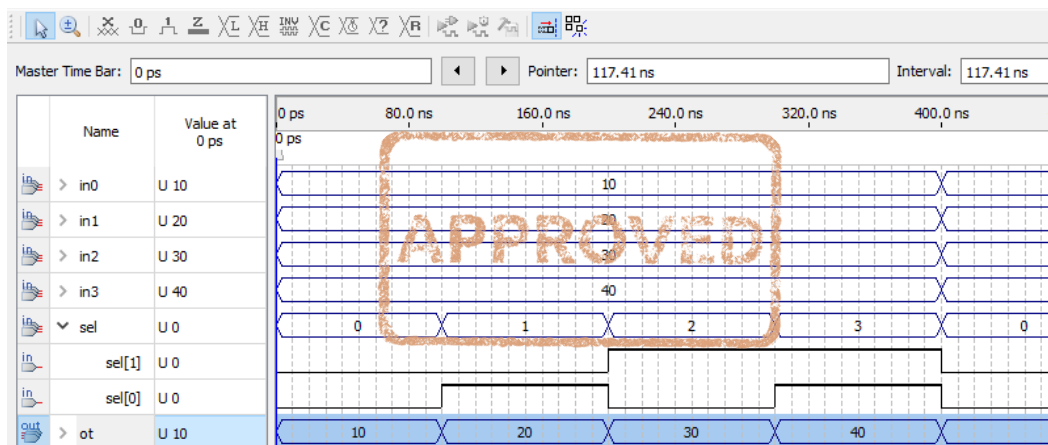


Figura 4 – Formas de onda de saída VWF - Mux.

Continua ...



## PARTE 2 – Descrição do DEMUX de 4 canais

1- No quartus II vá em **File->New->VHDL** para criar um novo arquivo VHDL e edit o código da Figura 5. Salve o arquivo com o nome **demux\_1\_4.vhd**. Esse código VHDL implementa a descrição do circuito demultiplexador de 4 canais com N bits.

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity demux_1_4 is
5      generic (N : integer := 8);
6      port (input : in STD_LOGIC_VECTOR (N-1 downto 0);
7            sel   : in STD_LOGIC_VECTOR (1 downto 0);
8            y0,y1,y2,y3 : out STD_LOGIC_VECTOR (N-1 downto 0));
9  end demux_1_4;
10
11  ----- usando when else -----
12  architecture rtl of demux_1_4 is
13      constant zero : STD_LOGIC_VECTOR (N-1 downto 0) := "00000000";
14
15  begin
16      y0 <= input when sel = "00" else zero;
17      y1 <= input when sel = "01" else zero;
18      y2 <= input when sel = "10" else zero;
19      y3 <= input when sel = "11" else zero;
20  end rtl;
21
22  ----- usando case-when -----
23  architecture comportamental of demux_1_4 is
24      constant zero : STD_LOGIC_VECTOR (N-1 downto 0) := x"0";
25      -- signal abc : STD_LOGIC_VECTOR (1 downto 0) := "00";
26  begin
27      process(sel,input)
28      begin
29          y0 <= zero; y1 <= zero; y2 <= zero; y3 <= zero;
30          case sel is
31              when "00" => y0 <= input;
32              when "01" => y1 <= input;
33              when "10" => y2 <= input;
34              when "11" => y3 <= input;
35              when others => null; --teste
36          end case;
37      end process;
38  end comportamental;
39
40  ----- usando if-then-else com process-----
41  architecture comportamental2 of demux_1_4 is
42      constant zero : STD_LOGIC_VECTOR (N-1 downto 0) := "00000000";
43
44  begin
45      process(sel,input) -- (sel,input,select_ch)
46      begin
47          if sel = "00" then y0 <= input;
48          else y0 <= zero;
49          end if;
50          if sel = "01" then y1 <= input;
51          else y1 <= zero;
52          end if;
53          if sel = "10" then y2 <= input;
54          else y2 <= zero;
55          end if;
56          if sel = "11" then y3 <= input;
57          else y3 <= zero;
58          end if;
59      end process;
60  end comportamental2;
```

Figura 5 – Código VHDL da descrição do DEMUX de 4 canais de N bits

2- Utilize as formas de onda de entrada da Figura 6 para simular o circuito implementado no código VHDL da Figura 5.

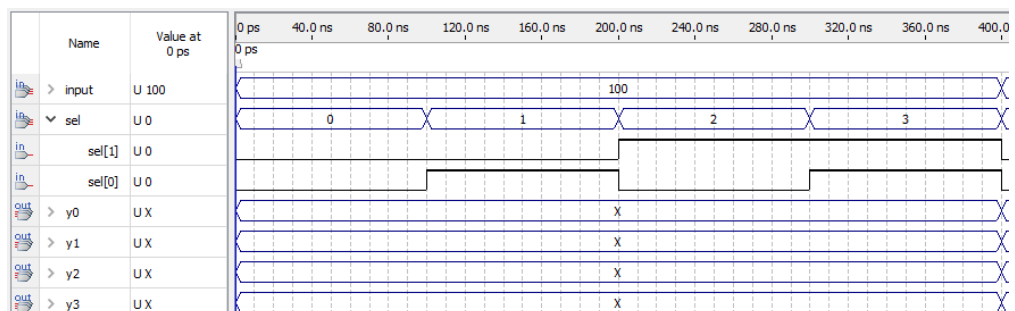


Figura 6 – Formas de onda de entrada VWF.

3- Realiza a simulação do circuito e veja as formas de onda de saída na ferramenta University Program VWF. Capture a tela de saída do resultado da simulação. A captura deve ser similar ao apresentado na Figura 7.

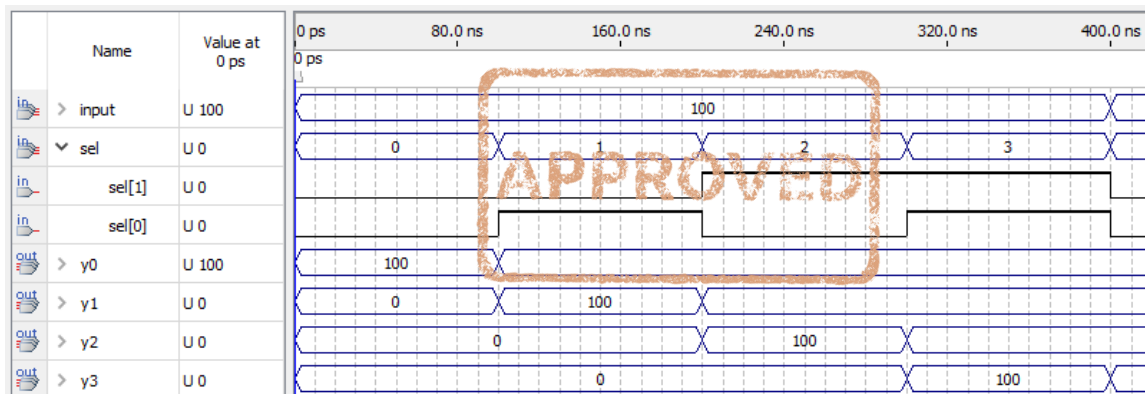


Figura 7 – Formas de onda de saída VWF - Demux.

### PARTE 3 – Descrição do MUX\_DEMUX de 4 canais

- 1- No quartus II vá em **File->New->VHDL** para criar um novo arquivo VHDL e edit o código da Figura 8. Salve o arquivo com o nome **mux\_demux\_4ch.vhd**. Esse código VHDL implementa a descrição do circuito apresentado na Figura 1 – **mux\_demux de 4 canais com N bits**.

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity mux_demux_4ch is
5  |
6  |   generic (N : integer := 8);
7  |
8  |   port (in0, in1, in2, in3 : in  STD_LOGIC_VECTOR (N-1 downto 0);
9  |         sel : in  STD_LOGIC_VECTOR (1 downto 0);
10 |         out0, out1, out2, out3 : out STD_LOGIC_VECTOR (N-1 downto 0));
11 | end mux_demux_4ch;
12
13 architecture estrutural of mux_demux_4ch is
14 | -- Criação do sinal interno da entidade
15 |   signal v : STD_LOGIC_VECTOR (N-1 downto 0);
16 |
17 | begin
18 | -- Cria uma instância de um mux_4_1 chamada mux com arquitetura rtl
19 |   mux:entity work.mux_4_1(rtl)
20 |     generic map (N => N)
21 |     port map (in0, in1, in2, in3, sel, v); -- método posicional
22 |
23 | -- Cria uma instância de um demux_1_4 chamada demux com arquitetura rtl
24 |   demux:entity work.demux_1_4(rtl)
25 |     generic map (N => N)
26 |     port map (v, sel, out0, out1, out2, out3); -- método posicional
27 |   --port map (y3 => out3, y0 => out0, y1 => out1, y2 => out2, sel => sel, input => v);
28 |   --opcionalmente poderia ter sido usado o método por atribuição direta
29 |
30 | end estrutural;
```

Figura 8 – Código VHDL da descrição do MUX\_DEMUX de 4 canais de N bits



- 1- Utilize as formas de onda de entrada da Figura 9 para simular o circuito implementado no código VHDL da Figura 8.

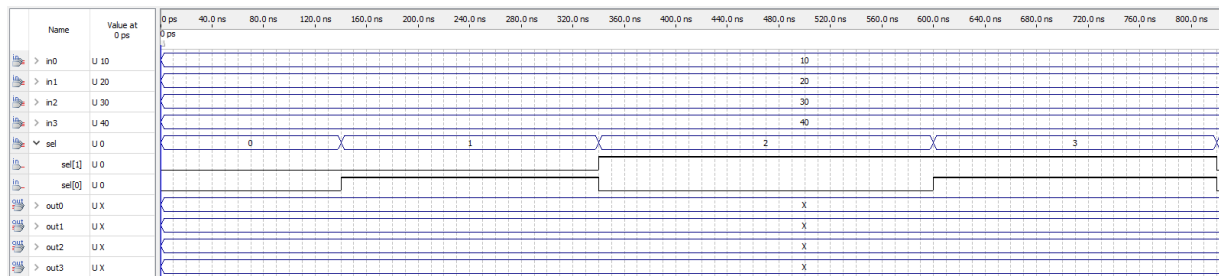


Figura 9 – Formas de onda de entrada VWF.

- 2- Realiza a simulação do circuito e veja as formas de onda de saída na ferramenta University Program VWF. Capture a tela de saída do resultado da simulação. A captura deve ser similar ao apresentado na Figura 10.

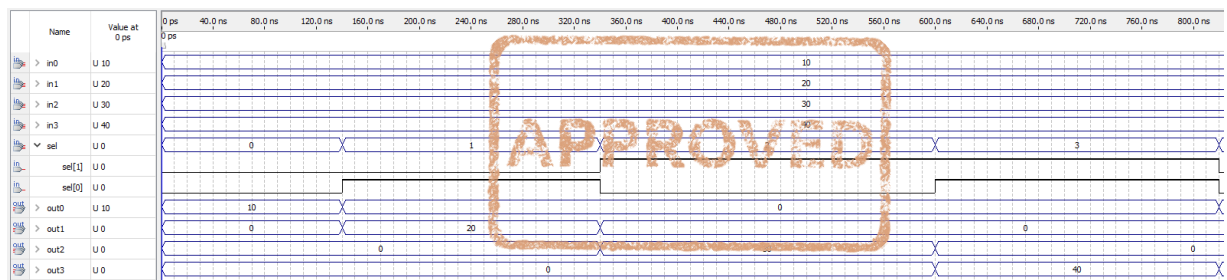


Figura 7 – Formas de onda de saída VWF – Mux\_Demux.

#### TAREFAS:

- 1- Modifique o quantidade de bits dos canais para 4 bits e refaça a simulação da entidade **mux\_demux\_4ch**;
- 2- Modifique a condição de saída para alta impedância de todos os canais do demux quando da condição não selecionado e refaça a simulação da entidade **mux\_demux\_4ch**;
- 3- Na descrição estrutural da entidade **mux\_demux\_4ch** modifique a definição da arquitetura a ser usada das entidades mux e demux ao instanciá-las.